

Lecture 52 : Part 3 : SQL Injection

Error Based from Web Application

Introduction

In the digital age, data is paramount, and its security is a constant battle. From personal information to corporate intellectual property, databases hold the keys to vast amounts of sensitive data. However, these digital vaults are not impenetrable. One of the most insidious threats they face is SQL Injection (SQLi), a technique that exploits vulnerabilities in web applications to manipulate backend databases. This lecture delves into a specific, powerful variant: Error-Based SQL Injection.

Imagine a scenario where a seemingly innocuous web form, designed to retrieve information, can be tricked into revealing the entire structure and contents of its underlying database. Error-based SQLi leverages the database's own error messages, which are often inadvertently displayed to the user, to systematically extract information. By carefully crafting malicious SQL queries, an attacker can provoke the database into revealing details about its schema, tables, columns, and even sensitive user credentials, one error message at a time.

This session provides a practical demonstration of how these techniques work, using the intentionally vulnerable Metasploitable 2 operating system as a target. Understanding these methods is crucial not only for aspiring ethical hackers and penetration testers but also for developers and security professionals who need to build robust defenses against such attacks. By dissecting the attacker's methodology, we gain invaluable insights into protecting our digital assets from sophisticated data breaches.

Core Concepts

Term	Definition	Significance
SQL Injection (SQLi)	A code injection technique used to attack data-driven applications, where malicious SQL statements are inserted into an entry field for execution.	It's a fundamental web vulnerability allowing unauthorized database access and manipulation.
	A type of SQLi where an attacker relies on error	It allows enumeration of database schema,

Term	Definition	Significance
Error-Based SQL Injection	messages generated by the database server to gather information about the database structure.	tables, and columns when direct output is not available.
Metasploitable 2	A deliberately vulnerable Linux virtual machine designed for security training and testing.	Provides a safe, controlled environment to practice and understand various attack techniques, including SQLi.
ORDER BY clause	A SQL clause used to sort the result-set of a query by one or more columns.	Can be exploited in error-based SQLi to determine the exact number of columns in a table by provoking an error.
UNION SELECT statement	A SQL operator used to combine the result-set of two or more SELECT statements into a single result-set.	Crucial for extracting data in SQLi by combining an attacker's SELECT query with a legitimate, often empty, query.
information_schema	A standard set of views in SQL databases that provides access to database metadata, such as table names, column names, and data types.	An invaluable resource for attackers to enumerate the database structure (tables, columns) once SQLi is achieved.
@@version	A system variable in some SQL databases (like MySQL) that returns the version of the database server.	Allows an attacker to identify the specific database software and version, which can inform further targeted exploits.

Term	Definition	Significance
user (function)	A SQL function that returns the current user and host information of the database connection.	Helps an attacker identify the privileges and context of the compromised database connection.
Enumeration	The process of systematically discovering and listing information about a target system, such as database names, table names, and column names.	A critical phase in SQLi, enabling attackers to map out the database structure before extracting sensitive data.

Detailed Analysis

Error-based SQL Injection is a powerful technique that leverages the database's own error messages to systematically uncover its structure and contents. This process typically involves two main phases: first, determining the number of columns in a target table, and second, using that information to extract data through `UNION SELECT` statements.

1. Discovering Column Count with `ORDER BY`

The `ORDER BY` clause is typically used in SQL queries to sort the result set based on one or more columns. For instance, `ORDER BY 1` sorts by the first column, `ORDER BY 2` by the second, and so on. An attacker can exploit this functionality to determine the exact number of columns in a table without knowing their names.

The methodology involves progressively increasing the column number in the `ORDER BY` clause until the database throws an error. If a query like `ORDER BY N` executes successfully, it implies that at least `N` columns exist. When `ORDER BY N+1` results in an error, it indicates that the table has exactly `N` columns.

Consider a scenario where a web application executes a query similar to this:

```
SELECT * FROM accounts WHERE username = 'test' AND password = 'any' ORDER BY 1;
```

If this query returns an empty set (meaning no matching user), it's a normal response. The attacker then increments the `ORDER BY` value:

- `ORDER BY 1` : No error, empty set.

- ORDER BY 2 : No error, empty set.
- ORDER BY 3 : No error, empty set.
- ORDER BY 4 : No error, empty set.
- ORDER BY 5 : No error, empty set.
- ORDER BY 6 : **Error!**

This error, often displayed to the user, signifies that the `accounts` table does not have a sixth column. Therefore, the attacker can deduce that the `accounts` table has exactly 5 columns. This crucial piece of information is the foundation for the next stage of the attack.

2. Data Extraction using UNION SELECT

Once the number of columns is known, the `UNION SELECT` statement becomes the primary tool for data extraction. The `UNION` operator combines the result sets of two or more `SELECT` statements. For `UNION` to work, both `SELECT` statements must have the same number of columns and compatible data types in corresponding positions.

The strategy here is to craft a `UNION SELECT` query that combines the original (often empty) result set with a `SELECT` statement designed to retrieve specific information. Since the initial query (e.g., `SELECT * FROM accounts WHERE username = 'test' AND password = 'any'`) is designed to return an empty set, the results from the `UNION SELECT` portion will be displayed. The attacker replaces some of the `SELECT` ed values with constants (like `1, 2, 3, 4, 5`) and strategically places functions or metadata queries in the columns that are likely to be displayed by the web application.

a. Extracting Database Version

To identify the database version, an attacker can inject the `@@version` system variable into one of the `SELECT` ed columns. Assuming 5 columns were identified:

```
SELECT * FROM accounts WHERE username = 'test' AND password = 'any' UNION SELECT 1, 2,
```

This query would return a result set where the fourth column displays the database version (e.g., `5.0.51a-3ubuntu5.4`). The other columns would show the placeholder values `1, 2, 3, 5`.

b. Extracting Current User

Similarly, to find out the current database user, the `user` function can be used:

```
SELECT * FROM accounts WHERE username = 'test' UNION SELECT 1, 2, 3, user, 5
```

The output would reveal the username and host, such as `root@localhost`, in the fourth column.

c. Enumerating Table Names

To discover table names within the database, attackers query the `information_schema.tables` view, which contains metadata about all tables. The `table_name` column from this view is extracted:

```
SELECT * FROM accounts WHERE username = 'test' UNION SELECT 1, 2, 3, table_name, 5 FROM
```

This query would return a list of all table names in the database, with each table name appearing in the fourth column of successive rows.

d. Enumerating Column Names

Once a target table is identified (e.g., `users_users`), an attacker can enumerate its columns by querying `information_schema.columns` and selecting the `column_name`:

```
SELECT * FROM accounts WHERE username = 'test' UNION SELECT 1, 2, 3, column_name, 5 FROM
```

This would list all column names belonging to the `users_users` table, again in the fourth column.

e. Extracting Specific Data

Finally, with knowledge of table and column names, an attacker can extract sensitive data directly. For example, to retrieve usernames and passwords from the `accounts` table, assuming they are in the third and fourth columns respectively:

```
SELECT * FROM accounts WHERE username = 'any' UNION SELECT 1, 2, username, password, 5
```

This query would display the usernames in the third column and their corresponding passwords in the fourth column, effectively dumping credentials from the database. This demonstrates the full power of error-based SQL injection to enumerate and extract critical information.

Key Takeaways

- [] Error-based SQL Injection exploits database error messages to gather information.
- [] The `ORDER BY` clause can be used to determine the exact number of columns in a table by provoking an error when an invalid column index is used.
- [] The `UNION SELECT` statement is critical for extracting data, requiring the same number of columns as the original query.
- [] An initial `SELECT` query designed to return an empty set (e.g., `WHERE username = 'test' AND password = 'any'`) allows the `UNION` ed data to be displayed.
- [] Database metadata, accessible via `information_schema.tables` and `information_schema.columns` , is crucial for enumerating table and column names.
- [] System functions like `@@version` and `user` can reveal valuable information about the database server and current user.
- [] Attackers can systematically extract sensitive data, such as usernames and passwords, once table and column names are identified.
- [] Metasploitable 2 serves as an excellent, controlled environment for practicing and understanding SQL Injection techniques.

Conclusion

The detailed exploration of error-based SQL Injection techniques underscores a critical reality in cybersecurity: seemingly minor vulnerabilities can be leveraged to devastating effect. By systematically exploiting database error messages and the `ORDER BY` and `UNION SELECT` clauses, an attacker can meticulously map out a database's structure and extract sensitive information, from system versions and user credentials to entire data sets. This process, demonstrated effectively on Metasploitable 2, highlights the ingenuity of attackers and the inherent risks in insecure web application development.

For students and professionals alike, understanding these attack vectors is not merely an academic exercise; it is an imperative for building resilient digital infrastructure. The ability to enumerate database schemas and extract data through error messages serves as a stark reminder that every input field in a web application is a potential gateway to the backend database. Moving forward, the focus must shift towards proactive defense mechanisms, including robust input validation, parameterized queries, and least privilege principles, to prevent such sophisticated attacks from ever reaching the database. The next step in this learning journey will

be to understand how these types of queries are actually injected and executed from a real-world web application context.